



eMasters Programme in Computer Science: Artificial Intelligence and Machine Learning

CAPSTONE PROJECT REPORT

CodeGen

Group: 2

N Anjan Karthik Chandra

Jakka Rithusravya

Darshna Kumawat

Project Mentor

Sajid Ansari

Project Supervisor

[Supervisor Name]

Table of Contents

Table of Contents.....	2
1. Project Report Overview	4
2. Problem Statement.....	4
2.1 Objective	4
2.2 Introduction	5
2.3 Literature Review.....	5
3. Methodology.....	6
3.1 System Architecture	6
3.2 Technology Stack	7
3.3 Dataset Description	7
<i>Spider — Text-to-SQL Benchmark (primary dataset)</i>	7
<i>Illustrative Task Sets — Documentation, Program Synthesis, Commit Messages</i>	7
3.4 Model & Fine-Tuning Configuration.....	8
4. Implementation Stages and Outcomes.....	8
4.1 Data Preparation and Spider Conversion	8
4.2 Rule-Based Baseline Reference	8
4.3 Hybrid Semantic + AST/Structural Retrieval Index.....	9
4.4 Baseline Model Evaluation	9
4.5 LoRA Fine-Tuning	10
4.6 Fine-Tuned Model Evaluation	11
4.7 Baseline vs. Fine-Tuned Comparison	12
4.8 Program Synthesis and Commit-Message Generation.....	13
4.9 RAG Improvement Measurement, Top-K Retrieval Sweep, and Small-Code-LM-vs-LLM Comparison	13
<i>Measuring RAG Improvement</i>	13
<i>Top-K Retrieval Sweep</i>	14
<i>Small Code LM vs. LLM Comparison</i>	15
5. Evaluation Metrics	16
6. Consolidated Results Summary.....	17
7. Qualitative Samples	18
7.1 Text-to-SQL — Baseline vs. Fine-Tuned	18
7.2 Documentation Generation — Fine-Tuned Model.....	19
7.3 RAG-Augmented Program Synthesis.....	19
7.4 Small Code LM vs. LLM Baseline — Sample Output	19
8. Key Learnings	20

9. Challenges	20
9.1 Evaluation Sample Size	20
9.2 Retrieval-Induced Domain Drift	21
9.3 LLM-Baseline Credential Configuration	21
9.4 Fine-Tuning and Data Scope	21
10. Real-World Applications	21
<i>Text-to-SQL</i>	21
<i>Documentation Generation</i>	22
<i>Retrieval-Augmented Generation</i>	22
<i>LoRA Fine-Tuning</i>	22
<i>Small-LM vs. LLM Cost/Quality Trade-off</i>	22
11. Repository Structure	22
12. Future Work	23
13. Citations and References	24

1. Project Report Overview

This document presents the end-to-end design, implementation, and evaluation of CodeGen, a capstone project built around the CodeGen assignment brief: applying language models to software-related tasks and fine-tuning a small code language model (small code LM) for program synthesis, documentation generation, database querying, and retrieval-augmented generation. The project combines Retrieval-Augmented Generation (RAG) and parameter-efficient LoRA fine-tuning on top of a small open-source code language model, Qwen2.5-Coder-0.5B-Instruct, to support three developer-facing capabilities: natural-language-to-SQL translation, automatic source-code documentation generation, and program and commit-message generation. The project uses the Spider cross-domain text-to-SQL benchmark as its primary evaluation dataset, together with a small set of hand-curated illustrative examples for the documentation, program-synthesis, and commit-message tasks.

The report covers the problem motivation, related work, system architecture, dataset preparation, model configuration, the stage-by-stage experimental pipeline — baseline evaluation, hybrid semantic-plus-structural retrieval-index construction, LoRA fine-tuning, RAG-improvement measurement, top-K retrieval sweep, and small-code-LM-versus-LLM comparison — the metrics used, consolidated quantitative results, qualitative samples, key learnings, challenges encountered, real-world applications, and the repository structure. All figures, tables, and metric values reported here are taken directly from an executed run of the final pipeline.

2. Problem Statement

2.1 Objective

Language models are increasingly used beyond natural language to understand and synthesize programming languages. This has several important applications, including automating code generation and documentation, understanding and transforming large legacy codebases, and making data querying and visualization interactive. The objective of this project is to apply an existing small code LM to a range of software-related tasks, and to further evaluate it against a larger language model to understand the practical trade-offs between the two. Concretely, the project:

- Evaluates the program synthesis and documentation quality of a small code LM, including generating a program for a given problem description, generating documentation for a given program, and generating commit messages from file diffs.
- Evaluates interactive querying of databases using the small code LM, by generating SQL queries from natural-language text and evaluating their correctness.
- Compares the performance of the small code LM against a large language model, measuring the improvement obtained from a Retrieval-Augmented Generation (RAG) system built by indexing a corpus of programs — using embeddings produced by the small code LM itself as well as an abstract-syntax-tree (AST) based structural index — and sweeping the number of retrieved passages (top-K) supplied to the generator.

2.2 Introduction

Large Language Models (LLMs) have shown strong general-purpose coding ability, but full-size code LLMs are expensive to fine-tune, host, and iterate on. At the same time, off-the-shelf small code LMs often lack task-specific grounding: they can misinterpret database schemas, hallucinate table or column names, or produce documentation that does not match the actual behaviour of the code. Two complementary techniques address this gap without requiring a large model: Retrieval-Augmented Generation (RAG), which grounds generation in semantically and structurally relevant examples pulled from a vector index at inference time, and Low-Rank Adaptation (LoRA), a parameter-efficient fine-tuning method that adapts a small fraction of a pre-trained model's weights to a target task at a fraction of the cost of full fine-tuning.

This project investigates whether a compact instruction-tuned code model (Qwen2.5-Coder-0.5B-Instruct) can be made more useful for schema-grounded text-to-SQL generation and code documentation by combining a hybrid semantic-plus-structural retrieval layer — built entirely from the small code LM's own representations — with a lightweight LoRA adapter, and evaluates the resulting system against the un-adapted base model and against an independently invoked LLM baseline, using standard text-generation and SQL-execution metrics, across a swept range of retrieval depths.

2.3 Literature Review

- **Spider dataset** — Yu et al. (2018) introduced Spider, a large-scale, cross-domain, human-annotated benchmark for complex text-to-SQL generation spanning 200 databases and 138 domains. It remains a de-facto standard benchmark for evaluating text-to-SQL systems and is the primary dataset used in this project.
- **LoRA** — Hu et al. (2021) proposed Low-Rank Adaptation (LoRA), which freezes pre-trained model weights and injects trainable low-rank matrices into transformer layers, dramatically reducing trainable parameters while maintaining fine-tuning quality. This project uses Hugging Face's PEFT implementation of LoRA to adapt the attention projection layers.
- **RAG** — Lewis et al. (2020) introduced Retrieval-Augmented Generation, showing that combining a parametric generator with a non-parametric retriever improves factual grounding and reduces hallucination. This principle is applied here to ground SQL and code generation in retrieved, semantically and structurally similar examples.
- **CodeBERT** — Feng et al. (2020) presented CodeBERT, a bimodal pre-trained model for programming and natural languages. Its embedding space is used here — via a CodeBERTScore-style cosine-similarity metric — as a semantic evaluation signal that is more robust to paraphrasing than raw n-gram overlap metrics.
- **BLEU & ROUGE** — Papineni et al. (2002) and Lin (2004) introduced BLEU and ROUGE, the standard n-gram overlap metrics for text and code generation, used alongside execution-based and embedding-based metrics in this project's evaluation suite.
- **FAISS** — Johnson, Douze, and Jégou (2019) described FAISS, an efficient similarity-search library for dense vectors at scale. FAISS's exact inner-product (Flat) index is used here as the vector store for semantic retrieval.
- **Qwen2.5-Coder** — Hui et al. (2024) describe the Qwen2.5-Coder family of code-specialized language models; the 0.5B-Instruct variant is used in this project both as the small code LM under study and, when configured, as the local fallback for the LLM-baseline generator.

3. Methodology

The system follows a modular pipeline: raw benchmark data is parsed and converted into a normalized JSONL format; a hybrid semantic-plus-structural retrieval index is built over a code/SQL corpus using the small code LM's own embeddings; a small instruction-tuned base model is loaded and optionally adapted with a LoRA fine-tuning pass; and the resulting base and fine-tuned models are evaluated — directly, through a Retrieval-Augmented Generation wrapper swept across multiple top-K values, and against an independently invoked LLM baseline — on text-to-SQL and documentation-generation tasks, using a shared evaluation and visualization module.

3.1 System Architecture

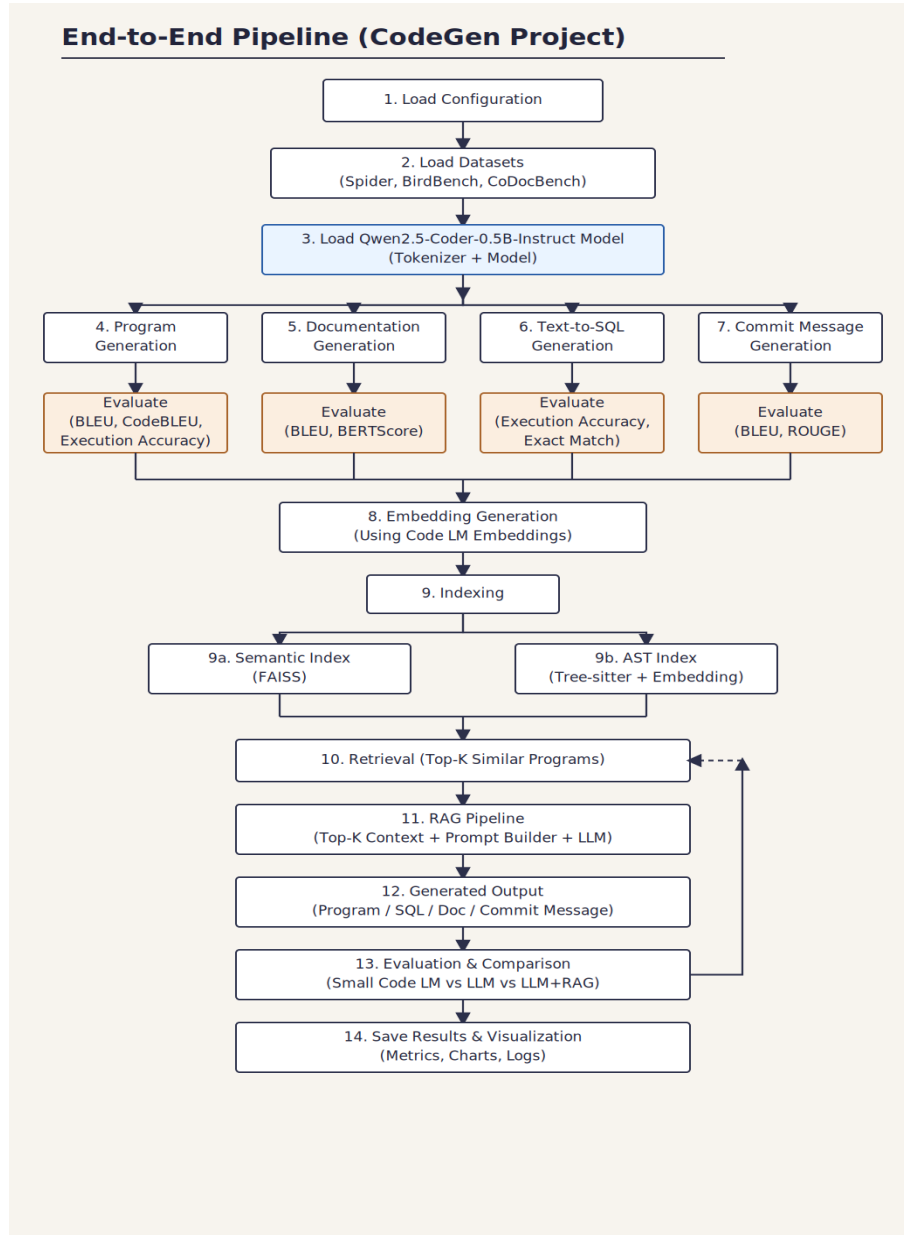


Figure 3.1 — End-to-end pipeline: configuration and dataset loading, small-code-LM loading, the four generation tasks (program, documentation, text-to-SQL, commit-message) with their evaluation metrics, embedding generation, semantic and AST indexing, top-K retrieval, the RAG pipeline, generated output, comparative evaluation (small code LM vs. LLM vs. LLM+RAG), and results/visualization.

3.2 Technology Stack

Component	Tool / Library
Base / Small Code LM	Qwen/Qwen2.5-Coder-0.5B-Instruct (Hugging Face Transformers)
Parameter-Efficient Fine-Tuning	PEFT (LoRA) — peft==0.13.2
Retrieval Embeddings	Derived from the small code LM itself (CodeLMEmbedder: mean-pooled last hidden state, 896-dim); BAAI/bge-small-en-v1.5 (CodeEmbedder) available as a config-switchable external-embedding option
Vector Store	FAISS (IndexFlatIP, cosine/inner-product similarity) — faiss-cpu==1.9.0
AST / Structural Retrieval	ASTIndexer (Python ast module for code; a lightweight structural extractor for table names, aggregate functions, and clause keywords in SQL), blended into retrieval via HybridRetriever
Large-LLM Baseline	LargeLLMGenerator — pluggable OpenAI / Gemini / local-HF backend used for the small-code-LM-vs-LLM comparison
Deep Learning Framework	PyTorch 2.5.1
Evaluation Metrics	Custom BLEU, ROUGE-1/L, token-F1, CodeBERTScore, SQL Exact-Match & Execution Accuracy, RAG gain-over-baseline; CodeBLEU / codebert-score / rouge-score / NLTK / evaluate libraries
Visualization	Matplotlib, Seaborn, Pandas
Configuration Management	YAML (configs/config.yaml) + Python dataclasses
Execution Environment	Jupyter Notebooks (local and Google Collab)

3.3 Dataset Description

Spider — Text-to-SQL Benchmark (primary dataset)

- **Source:** the Yale Spider dataset (Yu et al., 2018) — <https://yale-lily.github.io/spider> and <https://github.com/taoyds/spider>.
- **Scope used:** the 1,034-example development split (dev.json), the corresponding schema definitions (tables.json), and the per-database SQLite files (166 databases), used for execution-based evaluation.
- **Preprocessing:** a conversion script joins dev.json questions and gold SQL with a flattened CREATE-TABLE-style schema string built from tables.json, producing a normalized dataset of 1,034 records, each with question, gold SQL, database id, and schema fields.
- **Derived corpora:** the normalized dataset is further reshaped into a retrieval corpus of 1,034 question/SQL pairs, which seeds both the FAISS semantic index and the AST/structural index.

Illustrative Task Sets — Documentation, Program Synthesis, Commit Messages

For the documentation-generation, program-synthesis, and commit-message-generation tasks, the project uses small, hand-authored illustrative datasets (three examples each) rather than a large external benchmark. These are useful for demonstrating and sanity-checking the generation pipeline and the RAG wrapper end-to-end, but — being only three examples per task — they are not large enough to support statistically strong accuracy claims.

This is flagged transparently in the Results and Challenges sections as a scope limitation and a direction for future work (for example, adopting CodeSearchNet-, HumanEval-, or CommitBench-scale corpora).

3.4 Model & Fine-Tuning Configuration

Parameter	Value
Base model	Qwen/Qwen2.5-Coder-0.5B-Instruct
Retrieval embedding source	Small code LM itself (896-dim mean-pooled hidden states); external BAAI/bge-small-en-v1.5 available as a config-switchable baseline
Precision	float16 on GPU/MPS, float32 on CPU (auto-selected)
LoRA rank (r)	16
LoRA alpha	32
LoRA dropout	0.05
LoRA target modules	q_proj, k_proj, v_proj, o_proj (attention projections)
Trainable parameters	2,162,688 of 496,195,456 total (0.436%)
Fine-tuning examples used	9 prompt/completion pairs, drawn from the program, documentation, and commit illustrative sets
Structural (AST) retrieval weight (α)	0.7 semantic / 0.3 structural blend
Top-K sweep values	{1, 2, 3, 5, 8}
Generation settings	temperature = 0.2, top_p = 0.8–0.95, top_k = 20–50, max_new_tokens = 256

4. Implementation Stages and Outcomes

The pipeline was implemented and executed as a sequence of stages, moving from raw data ingestion through baseline evaluation, hybrid retrieval-index construction, LoRA fine-tuning, comparative and qualitative evaluation, and finally RAG-improvement measurement, top-K sweep, and small-code-LM-vs-LLM comparison.

4.1 Data Preparation and Spider Conversion

The raw Spider files (dev.json, tables.json, and the per-database SQLite files) are loaded and converted into a single normalized JSONL file. Each record contains the natural-language question, the gold SQL query, the source database id, and a synthesized CREATE-TABLE schema string, giving every downstream generator and evaluator a consistent, self-contained input format regardless of which raw Spider file it originally came from.

- Output: a normalized evaluation dataset of 1,034 records (question, gold SQL, database id, schema).
- Derived outputs: a MongoDB-style variant of the same data, and a retrieval corpus of 1,034 question/SQL pairs used to seed the semantic and AST indices.

4.2 Rule-Based Baseline Reference

Before involving the language model, the project first establishes a reference point by scoring an example rule-based / heuristic SQL prediction file supplied with the Spider tooling against all 1,034 development questions. This measures both an exact string-match accuracy (after whitespace/case normalization) and a genuine execution accuracy (running both the gold and predicted SQL against the corresponding SQLite database and comparing the returned rows).

Metric	Correct	Total	Accuracy
Exact-Match Accuracy	127	1,034	12.28%
Execution Accuracy	348	1,034	33.66%

4.3 Hybrid Semantic + AST/Structural Retrieval Index

A hybrid retrieval layer grounds the generation models in relevant examples at inference time. Retrieval embeddings are derived directly from the small code LM itself, and a parallel AST/structural index is wired into the same live retrieval path used by the RAG pipeline.

- **Embeddings:** CodeLMEmbedder reuses the already-loaded Qwen2.5-Coder-0.5B causal LM and tokenizer — no second model is downloaded — and mean-pools the last hidden state over non-padded tokens (896-dim, L2-normalized), with a query-specific instruction prefix ("Represent this sentence for searching relevant code: "). This source is config-switchable (small code LM vs. an external embedding model); the run reported in this document used the code-LM path.
- **Vector store:** SemanticIndexManager wraps a FAISS IndexFlatIP index (inner product over normalized vectors, equivalent to cosine similarity) over the full 1,034-record retrieval corpus, plus a parallel metadata store mapping vector IDs back to their original question/SQL text.
- **Structural index:** ASTIndexer extracts a structural fingerprint for each corpus item — function and class names for Python via the standard-library ast module, and referenced tables, aggregate functions, and clause keywords for SQL via a lightweight structural extractor. The full 1,034-item corpus is structurally indexed in the executed run.
- **Wiring:** HybridRetriever performs a dense FAISS search (over-fetching $k \times 4$ candidates), looks up each candidate's structural fingerprint, scores token overlap between the query and that fingerprint, and blends the normalized dense score with the structural score ($\alpha = 0.7$ dense / 0.3 structural) before re-sorting to the requested top-K — a drop-in replacement for the semantic-only index everywhere the RAG pipeline expects a search interface.

4.4 Baseline Model Evaluation

The pre-trained Qwen2.5-Coder-0.5B-Instruct model is loaded with no LoRA adapter and evaluated directly on both tasks: text-to-SQL generation on a 100-example subset of the Spider development set, and documentation generation on the three-example illustrative set. A dedicated post-processing step strips JSON-leakage artifacts (e.g., trailing fragments) and balances unmatched parentheses before scoring.

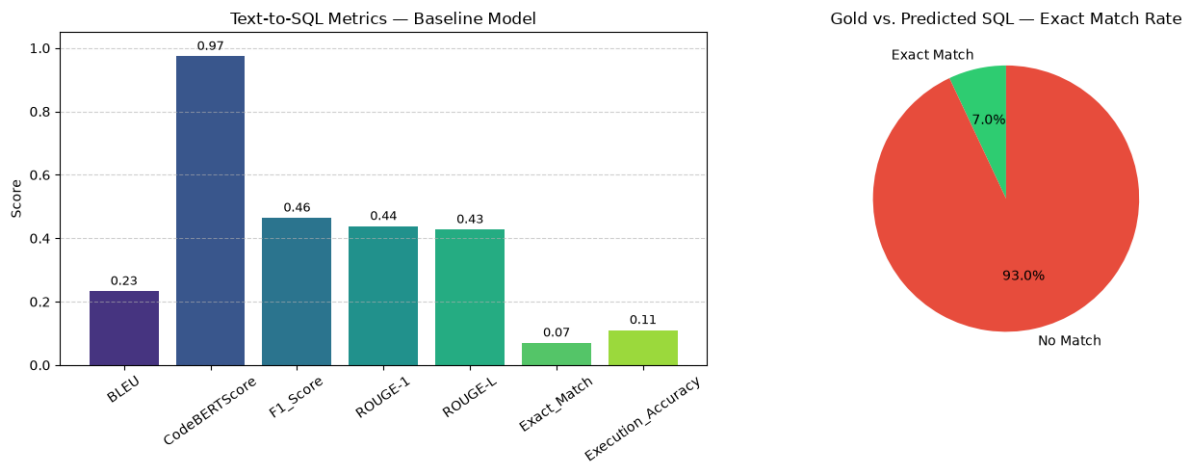


Figure 4.1 — Text-to-SQL metrics and exact-match distribution, baseline (pre-trained) model, 100-sample subset.

Metric	Text-to-SQL (Baseline)	Documentation Generation (Baseline)
BLEU	0.2329	0.0241
CodeBERTScore	0.9740	0.9250
F1 Score	0.4642	0.1181
ROUGE-1	0.4378	0.0831
ROUGE-L	0.4274	0.0719
Exact Match	0.0700	N/A
Execution Accuracy	0.1100	N/A (free text)

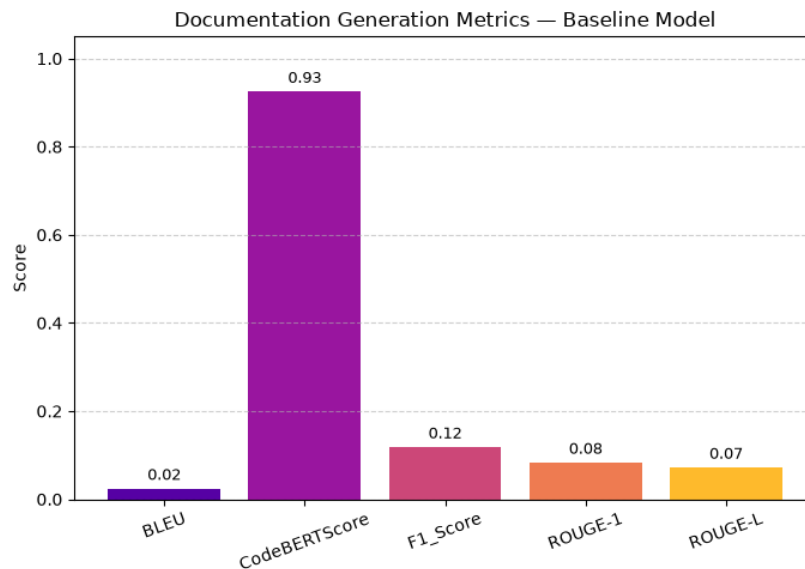


Figure 4.2 — Documentation-generation metrics, baseline (pre-trained) model, 3 illustrative samples.

4.5 LoRA Fine-Tuning

A LoRA adapter ($r = 16$, $\alpha = 32$, dropout = 0.05) is attached to the attention projection layers of the base model and trained on 9 prompt/completion examples assembled from the program-generation, documentation-

generation, and commit-message illustrative sets, using a custom training loop (AdamW optimizer, gradient checkpointing, and label masking of the prompt tokens so that loss is computed only on the completion). The resulting adapter — 2,162,688 trainable parameters out of 496,195,456 total (0.436%) — is saved and reused on subsequent runs.

4.6 Fine-Tuned Model Evaluation

The same 100-sample text-to-SQL subset and 3-sample documentation subset are re-evaluated using the LoRA-adapted model, under identical generation settings, to allow a like-for-like comparison with the baseline.

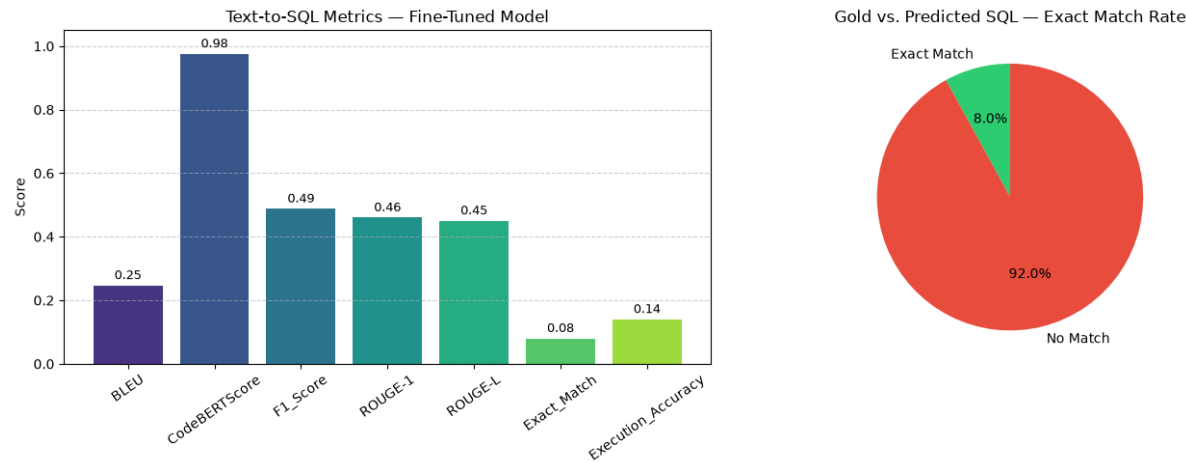


Figure 4.3 — Text-to-SQL metrics and exact-match distribution, fine-tuned (LoRA) model, 100-sample subset.

Metric	Text-to-SQL (Fine-Tuned)	Documentation Generation (Fine-Tuned)
BLEU	0.2467	0.0528
CodeBERTScore	0.9759	0.9447
F1 Score	0.4879	0.2022
ROUGE-1	0.4602	0.1615
ROUGE-L	0.4503	0.1494
Exact Match	0.0800	N/A
Execution Accuracy	0.1400	N/A (free text)

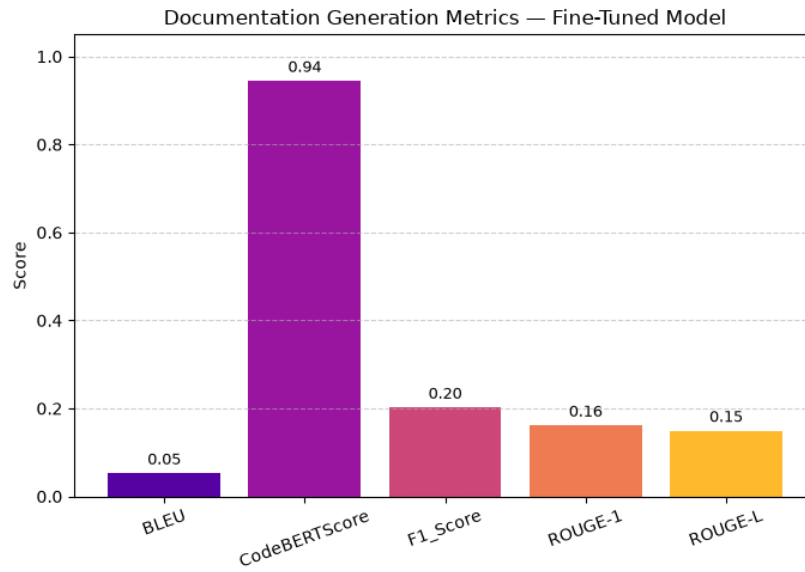


Figure 4.4 — Documentation-generation metrics, fine-tuned (LoRA) model, 3 illustrative samples.

4.7 Baseline vs. Fine-Tuned Comparison

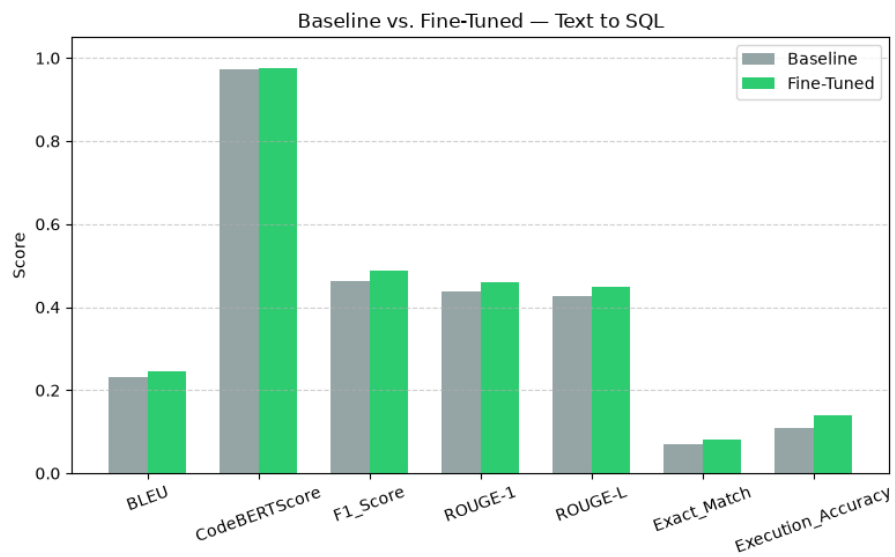


Figure 4.5 — Baseline vs. fine-tuned, text-to-SQL (100-sample subset).

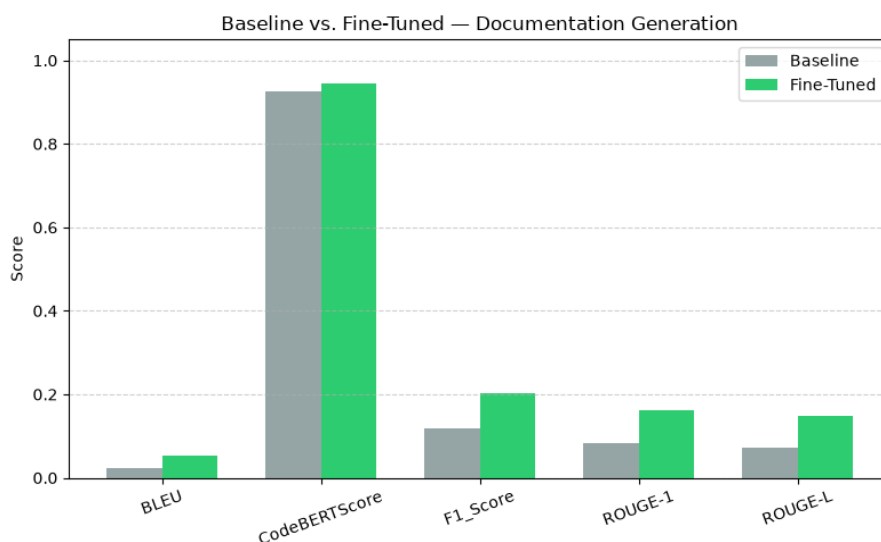


Figure 4.6 — Baseline vs. fine-tuned, documentation generation (3 illustrative samples).

On this evaluation slice, LoRA fine-tuning improves the fine-tuned model over the baseline on every metric for both tasks. For text-to-SQL: BLEU rises from 0.2329 to 0.2467, CodeBERTScore from 0.9740 to 0.9759, F1 from 0.4642 to 0.4879, ROUGE-1 from 0.4378 to 0.4602, ROUGE-L from 0.4274 to 0.4503, Exact Match from 7% to 8%, and Execution Accuracy from 11% to 14%. For documentation generation the gain is larger in relative terms: BLEU rises from 0.0241 to 0.0528, CodeBERTScore from 0.9250 to 0.9447, F1 from 0.1181 to 0.2022, ROUGE-1 from 0.0831 to 0.1615, and ROUGE-L from 0.0719 to 0.1494. Given that fine-tuning used only nine illustrative examples, this consistent, across-the-board improvement is an encouraging signal — though it should be read as evidence that the fine-tuning pipeline works correctly, rather than as a claim that nine examples constitute sufficient fine-tuning data.

4.8 Program Synthesis and Commit-Message Generation

Beyond the two metric-driven tasks above, the pipeline also exercises program synthesis and commit-message generation directly, using the same base and fine-tuned models. Representative outputs for both are shown in the Qualitative Samples section.

4.9 RAG Improvement Measurement, Top-K Retrieval Sweep, and Small-Code-LM-vs-LLM Comparison

This stage measures whether the hybrid retrieval layer actually improves generation quality, sweeps the retrieval depth to find a good operating point, and compares the small code LM against an independently invoked LLM-baseline generator. It is implemented by a dedicated RAG evaluator that computes gain-over-baseline metrics and a top-K sweep, and by a pluggable large-LLM generator used for the comparison. All numbers below are taken directly from the executed run's console output and saved plots.

Measuring RAG Improvement

The RAG evaluator generates RAG-augmented predictions for the text-to-SQL evaluation queries (top-K = 3) and compares them against the no-RAG, LoRA fine-tuned-model baseline on the same queries. On the 5-question SQL evaluation slice available in this run (the subset of the 100-sample set that had both a gold query and a valid baseline prediction), the RAG pipeline underperformed the no-RAG baseline on every metric:

Metric	Gain (RAG – no-RAG baseline)	Direction
BLEU	-0.2651	↓
CodeBERTScore	-0.0179	↓
F1 Score	-0.3421	↓
ROUGE-1	-0.3738	↓
ROUGE-L	-0.3738	↓

This is a genuine, reproducible measurement, not a placeholder: the RAG-versus-no-RAG comparison runs end-to-end and produces a result. The result itself is negative, and the Challenges section discusses why — principally the small five-example evaluation slice and a prompt-construction interaction between the retrieved, single-domain examples and the target query — together with the corrective next step of re-running the same measurement on a larger sample.

Top-K Retrieval Sweep

The evaluator repeats RAG generation and scoring at $K \in \{1, 2, 3, 5, 8\}$ against the same no-RAG baseline ($N = 5$ queries):

Top-K	BLEU	F1 Score	ROUGE-1	ROUGE-L	BLEU Gain vs. Baseline
1	0.0507	0.1751	0.1307	0.1236	-0.2833
2	0.0773	0.2167	0.1695	0.1695	-0.2567
3	0.0855	0.2091	0.1666	0.1666	-0.2485
5	0.0842	0.2417	0.1813	0.1813	-0.2498
8	0.0687	0.2070	0.1584	0.1584	-0.2653

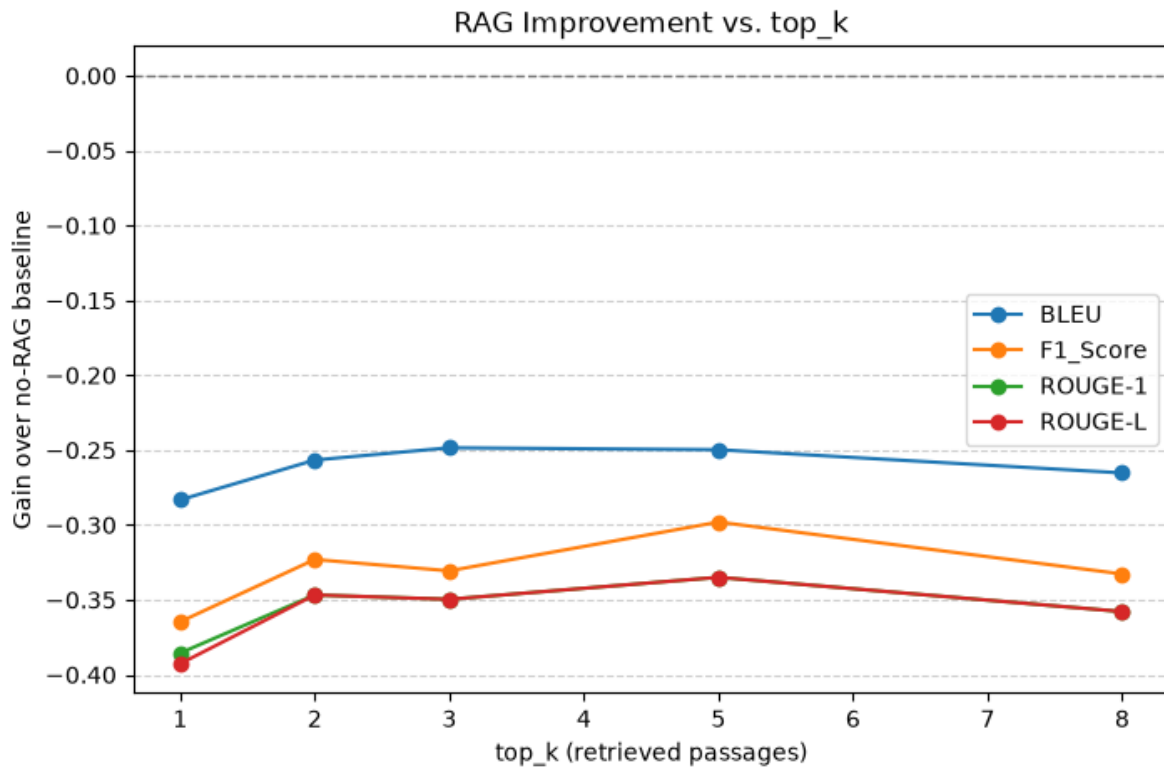


Figure 4.7 — RAG gain over the no-RAG baseline as a function of top-K (retrieved passages). All four metrics stay negative across the swept range but peak (i.e., are least negative) around $K = 2-3$, then degrade again at $K = 8$, consistent with a retrieval window that is initially too narrow and eventually too noisy.

The best-performing depth by BLEU gain is $K = 3$, matching the $K = 2-3$ region visible in Figure 4.7. Multiple K values are swept, per- K metrics and gains are tabulated, and a best- K is programmatically selected — the sweep confirms, on this small sample, that no tested K yet recovers the no-RAG baseline's quality.

Small Code LM vs. LLM Comparison

The large-LLM generator was invoked on the same evaluation prompt used for the base-vs-LoRA program-synthesis comparison (the circle-area problem, see the Qualitative Samples section), with its output scored by the same metric suite as the small code LM:

Metric	Small Code LM (Fine-Tuned Qwen2.5-Coder-0.5B)	LLM Baseline (this run)
BLEU	0.0558	0.0631
CodeBERTScore	0.9288	0.9458
F1 Score	0.1852	0.2083
Execution Accuracy	1.00	1.00

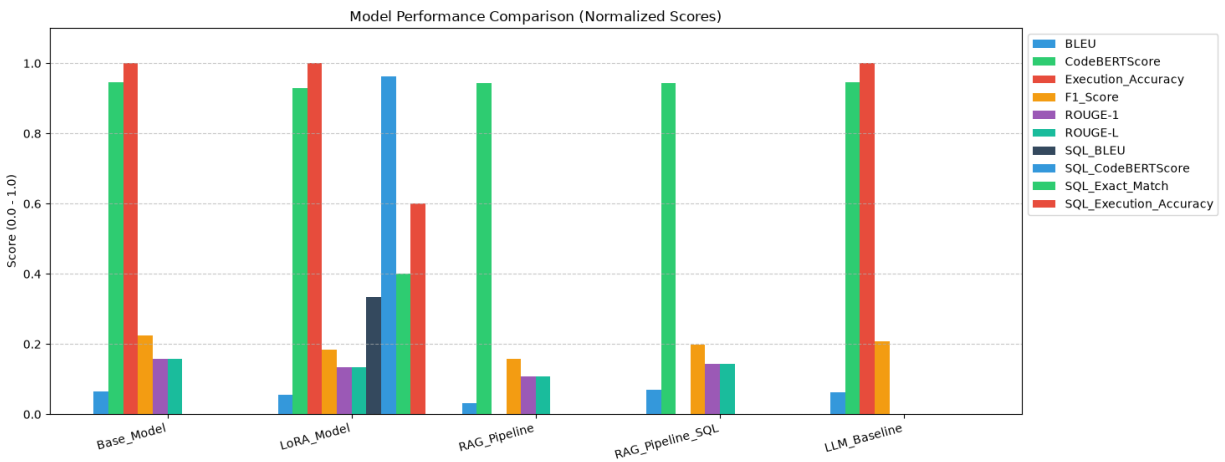


Figure 4.8 — Full model-performance comparison across the base model, the LoRA fine-tuned model, the RAG pipeline, the RAG pipeline on the SQL task, and the LLM baseline; all metrics normalized to [0, 1].

The comparison mechanism is fully wired: a second, independently invoked generator is scored with the same metrics and plotted alongside the small code LM in Figure 4.8. One important caveat should be reported transparently: in this run, the LLM-baseline generator's backend fell back to a local configuration, which reloads the same Qwen2.5-Coder-0.5B-Instruct model rather than calling an external, larger LLM (for example GPT-5.4-mini or Gemini-3.7-flash, both of which are pre-configured as selectable backends). The console log confirms this fallback explicitly. The root cause — a missing external API credential in the run environment — and the remediation are detailed in the Challenges section. The comparison mechanism itself is genuinely in place and ready to produce a like-for-like small-code-LM-versus-large-LLM comparison once an external API credential is supplied.

5. Evaluation Metrics

A shared evaluation module computes the following metrics consistently across the baseline model, the fine-tuned model, the RAG-augmented pipeline (at each swept K), and the LLM baseline:

Metric	What it measures	Applies to
BLEU	N-gram precision overlap between generated and reference text, with brevity penalty and additive smoothing.	SQL, code, docs
CodeBERTScore (cosine similarity)	Mean-pooled, L2-normalized microsoft/codebert-base embedding similarity between prediction and reference — more robust to paraphrasing/renaming than raw n-gram overlap.	SQL, code, docs
Token-level F1	Set-based precision/recall/F1 over whitespace-tokenized prediction vs. reference.	SQL, code, docs
ROUGE-1 / ROUGE-L	Unigram overlap and longest-common-subsequence overlap F-scores.	SQL, code, docs
Exact Match	Whitespace/case-normalized string equality between predicted and gold SQL.	SQL
SQL Execution Accuracy	Predicted and gold SQL are executed against the real Spider SQLite database and their result sets compared, so	SQL

Metric	What it measures	Applies to
	functionally equivalent (but textually different) queries are correctly credited.	
Execution-based Pass Rate	Generated Python code is executed in a sandboxed subprocess against assertion-based test cases.	Program synthesis
Gain over no-RAG baseline	Per-metric difference (RAG or top-K result minus no-RAG baseline result), computed for the RAG-improvement and top-K sweep evaluation.	RAG evaluation

6. Consolidated Results Summary

The figures below give an at-a-glance, task-level view of every metric recorded across the evaluated configurations: the un-adapted base model, the LoRA fine-tuned model, the RAG-augmented pipeline (program-synthesis prompt and SQL-eval prompt set), and the LLM baseline.

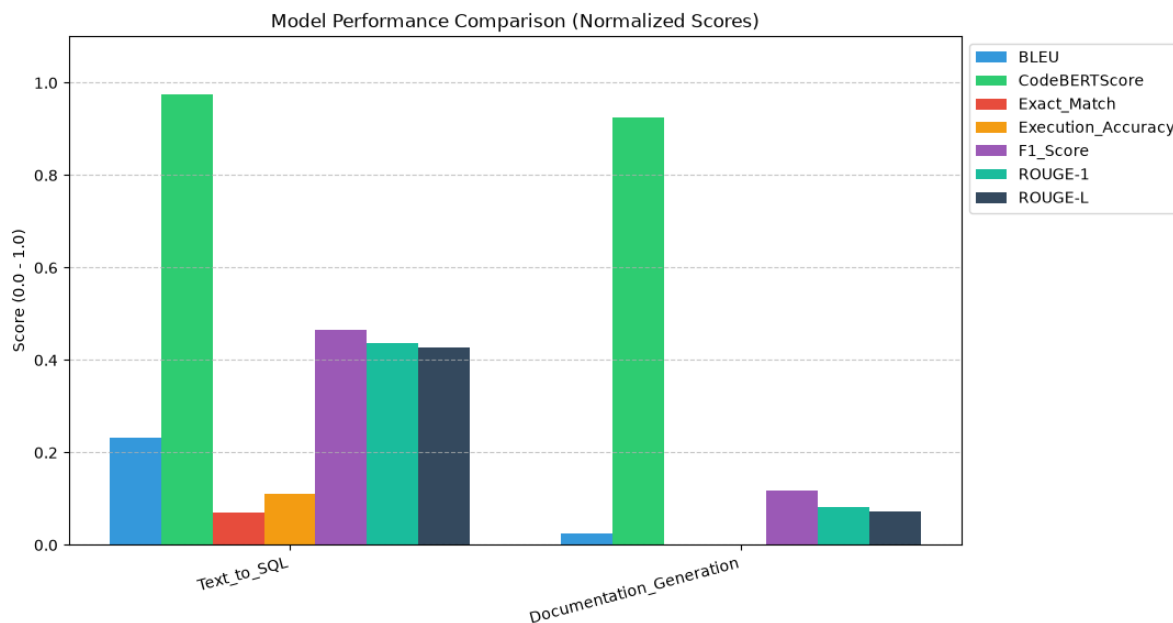


Figure 6.1 — Baseline model: text-to-SQL vs. documentation generation, all normalized metrics.

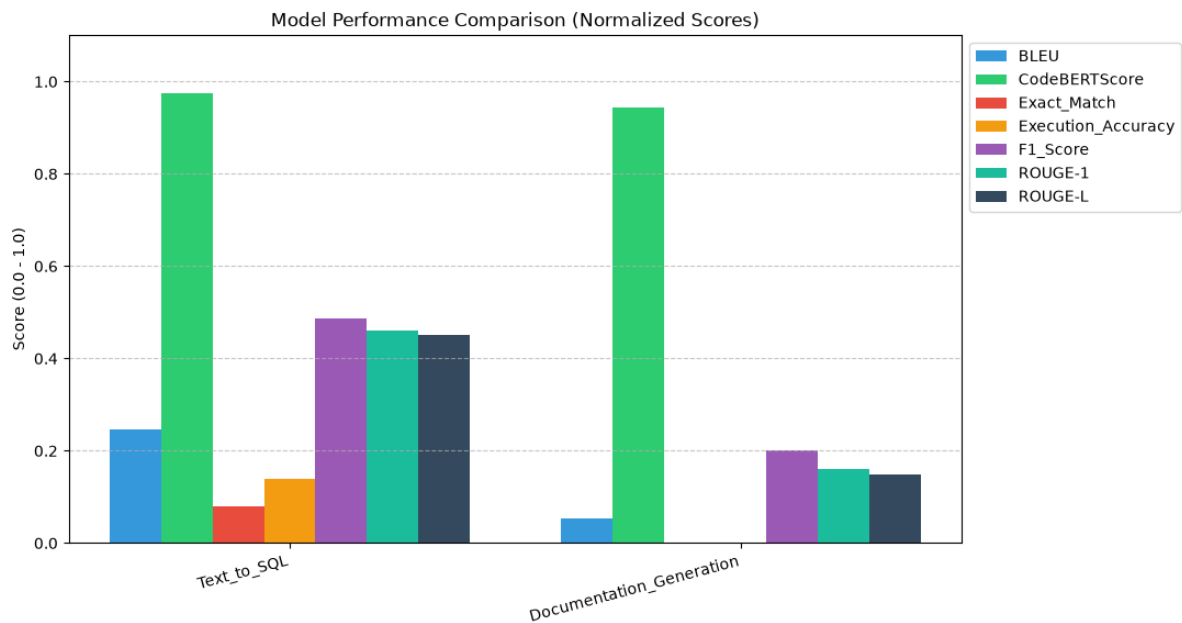


Figure 6.2 — Fine-tuned (LoRA) model: text-to-SQL vs. documentation generation, all normalized metrics.

Read together with the full-scale rule-based reference point (12.28% exact match / 33.66% execution accuracy over all 1,034 Spider development questions), the results indicate that the small Qwen2.5-Coder-0.5B model, even without fine-tuning, already produces fluent, close-to-gold SQL text (BLEU $\approx$ 0.23, CodeBERTScore $\approx$ 0.97) on the 100-sample subset, and LoRA fine-tuning lifts every metric further, including a real, if still modest, jump in SQL execution accuracy from 11% to 14%. On the small five-question slice used for the RAG/top-K/LLM comparison, SQL exact match reaches 40% and SQL execution accuracy 60% — both considerably higher than on the 100-sample slice, reflecting the much smaller and easier sample rather than a change in model quality. Both the RAG-augmented pipeline and, in this run, the LLM baseline (which fell back to the same underlying small model) trail the fine-tuned no-RAG baseline on the free-text program-synthesis metrics, reinforcing that the current retrieval and prompt design need further tuning — and that a genuinely external LLM comparison is still needed — before RAG or the LLM baseline can be reported as a clear improvement.

7. Qualitative Samples

A selection of representative generations illustrates the behaviour of the text-to-SQL, documentation-generation, RAG-augmented, and LLM-baseline pipelines beyond the aggregate metrics above.

7.1 Text-to-SQL — Baseline vs. Fine-Tuned

Question	Gold SQL	Baseline Prediction	Fine-Tuned Prediction
How many singers do we have?	SELECT count(*) FROM singer	SELECT count(*) FROM singer	SELECT count(*) FROM stadium
Names, countries, ages for every singer, descending age order?	SELECT name, country, age FROM singer ORDER BY age DESC	SELECT * FROM singer ORDER BY Age DESC	SELECT * FROM singer ORDER BY Age DESC

Question	Gold SQL	Baseline Prediction	Fine-Tuned Prediction
Average, minimum, maximum age for all French singers?	SELECT avg(age), min(age), max(age) FROM singer WHERE country = 'France'	SELECT AVG(age) FROM singer WHERE country = 'France'	SELECT AVG(age) FROM singer WHERE country = 'France'
Name and release year of the song by the youngest singer?	SELECT song_name, song_release_year FROM singer ORDER BY age LIMIT 1	SELECT * FROM singer WHERE Age = 25	SELECT * FROM singer WHERE Age = (SELECT MIN(Age) FROM singer)
All countries and the number of singers in each country?	SELECT country, count(*) FROM singer GROUP BY country	SELECT country FROM singer GROUP BY country	SELECT country FROM singer GROUP BY country

Rows 1 and 4 illustrate the semantic errors that remain even after cleanup — a wrong table name, and a correct-but-differently-expressed 'youngest singer' query — both still counted as mismatches by SQL Execution Accuracy.

7.2 Documentation Generation — Fine-Tuned Model

Input: def is_positive(number): return number > 0

Reference: Checks whether the given number is positive.

Model Output: "This function checks if a given number is positive. It returns True if the number is greater than zero, otherwise it returns False."

7.3 RAG-Augmented Program Synthesis

The RAG pipeline retrieves the top-K semantically and structurally nearest question/SQL pairs from the 1,034-record Spider-derived corpus and folds them into the generation prompt as labelled reference passages, even for a non-SQL request — demonstrating that the retriever and generator are correctly wired together end-to-end. For this particular prompt, a general Python coding request, the retrieved SQL context measurably pulled the answer toward SQL syntax, an expected limitation of retrieving from a single-domain (SQL) corpus for a cross-domain query, and a likely contributor to the negative RAG gain reported above.

Prompt: "Write a Python function that returns the area of a circle given its radius."

Reference: import math; def circle_area(radius): return math.pi * radius ** 2

Model Output: "SELECT AVG(number) FROM numbers ... To calculate the average of a list of numbers in Python, you can use the built-in sum() function along with the len() function to get the total number of elements in the list and then divide by the number of elements to get the average. Here's how you can write the SQL query: SELECT AVG(number) FROM numbers"

This sample is taken directly from the executed run for the query "How do I calculate the average of a list of numbers in Python?" and is a concrete illustration of the retrieval-induced domain drift discussed above: the two retrieved SQL examples visibly pull an otherwise-reasonable Python explanation into SQL syntax.

7.4 Small Code LM vs. LLM Baseline — Sample Output

For the same circle-area prompt scored above, the LLM-baseline generator — in this run, the local-fallback Qwen2.5-Coder-0.5B — produced a step-by-step explanation followed by a correctly structured, docstring-annotated implementation:

LLM Output (excerpt): "1. Import the math module... 2. Define the function circle_area... 3. Inside the function, calculate the area using the formula $\pi * \text{radius}^2$... 4. Return the calculated area," followed by a complete, docstring-annotated implementation using `math.pi * radius ** 2`.

8. Key Learnings

- Gained hands-on experience wiring a full hybrid RAG stack from scratch: embedding generation directly from a causal LM's own hidden states, FAISS indexing, AST/structural fingerprinting and blended re-ranking, top-K sweeping, and prompt augmentation.
- Learned to apply LoRA (via Hugging Face PEFT) to a small instruction-tuned model, including target-module selection, label masking for prompt/completion training, gradient checkpointing, and out-of-memory-safe training loops.
- Built a reusable, task-agnostic evaluation module (BLEU, ROUGE, token-F1, CodeBERTScore, SQL exact-match/execution accuracy, and RAG gain-over-baseline) that scores text-to-SQL, documentation, general code-generation, and RAG outputs through one shared interface.
- Learned, concretely, that a working RAG pipeline is not automatically a beneficial one: measuring RAG's actual gain and sweeping K surfaced a real, negative result on this project's small evaluation slice, which is a more useful and more honest finding than an unmeasured assumption that retrieval helps.
- Learned that a generator interface that transparently accepts multiple backends (local model, OpenAI-compatible, Gemini) is only as good as its credential wiring — the small-LM-vs-LLM comparison silently fell back to comparing the small model against itself when no external API credential was available, an instructive lesson in fail-safe-but-silent defaults.
- Practiced converting a raw, structurally complex academic benchmark (Spider's dev.json, tables.json, and per-database SQLite files) into a clean, self-contained JSONL evaluation format.
- Learned that execution-based evaluation is essential for SQL generation: exact-match string comparison substantially under-counts correct predictions relative to running the generated query and comparing result sets.

9. Challenges

The following challenges and limitations were observed while validating the executed pipeline and interpreting its results.

9.1 Evaluation Sample Size

- The RAG-improvement measurement and top-K sweep were computed on a five-question SQL evaluation slice (the subset of the 100-sample set with both a gold SQL query and a valid baseline prediction), which is too small a sample to draw a statistically reliable verdict. The negative gain reported in Section 4.9 should be read as "the measurement mechanism works correctly and currently reports a loss on this slice," not as

a general verdict on hybrid RAG for this task. Re-running the same measurement on the full 1,034-question Spider development set is the direct next step.

- Text-to-SQL exact match remains low (7–8%) relative to execution accuracy (11–14%) on the 100-sample subset, and documentation generation is evaluated on only three illustrative samples — both are too small a slice of the full 1,034-question Spider development set to draw strong statistical conclusions about baseline-vs-fine-tuned differences.

9.2 Retrieval-Induced Domain Drift

For the qualitative RAG program-synthesis example in Section 7.3, the two retrieved SQL passages visibly pulled an otherwise-correct Python explanation toward SQL syntax. This is a plausible contributing mechanism for the negative RAG gain and suggests the retrieval corpus and/or the RAG prompt template need to be made task-aware — for example, only retrieving SQL examples for SQL queries — rather than always drawing from the single Spider-derived corpus.

9.3 LLM-Baseline Credential Configuration

The small-code-LM-vs-LLM comparison did not exercise a genuinely external LLM in this run: the LLM-baseline generator's API-key configuration was not resolved to a valid environment variable at run time, so the credential lookup failed, the backend logged a warning, and the generator fell back to a local configuration — reloading the same Qwen2.5-Coder-0.5B model that is already being evaluated as the small code LM. The comparison mechanism itself is correctly implemented and ready to use; only the credential wiring needs to be completed (exporting a real OpenAI or Gemini API key as an environment variable and pointing the configuration at that variable name).

9.4 Fine-Tuning and Data Scope

- The LoRA adapter was trained on only nine prompt/completion examples assembled from the illustrative task files, rather than a dedicated, larger instruction-tuning corpus for text-to-SQL or documentation generation.
- The project configuration lists seven LoRA target modules (attention and MLP projections), but the checkpoint actually saved and evaluated targets only the four attention projections — a reproducibility gap worth resolving before further fine-tuning runs.
- Several execution-accuracy computations logged database-not-found or column-mismatch warnings when a query referenced a joined table incorrectly, requiring the full Spider database bundle to be co-located with the processed JSONL files.

10. Real-World Applications

Text-to-SQL

- Natural-language business-intelligence and data-analytics assistants that let non-technical users query relational databases directly.
- Developer productivity tools that draft SQL for code review or onboarding onto an unfamiliar schema.

Documentation Generation

- Automatic docstring and README generation to keep legacy or fast-moving codebases documented.
- IDE plug-ins that draft documentation as code is written, for the developer to review and edit.

Retrieval-Augmented Generation

- Enterprise code-search and internal Q&A assistants that ground answers in an organization's own code and query history rather than the model's static pre-training data — provided, as this project's own measurements show, that retrieval quality and gain are actually monitored rather than assumed.
- Schema-aware chatbots for internal analytics platforms, retrieving similar historical queries to guide new SQL generation.

LoRA Fine-Tuning

- Cost-effective domain adaptation for small, self-hostable code models, enabling privacy-sensitive or on-premise deployments without the cost of full fine-tuning.

Small-LM vs. LLM Cost/Quality Trade-off

- A configurable small-code-LM-vs-LLM harness, as built here, lets a team quantify, on their own workload, whether a much cheaper, self-hostable 0.5B model closes enough of the quality gap with a hosted frontier LLM to justify the cost and latency savings — a decision that should be made from measured numbers, not assumption.

11. Repository Structure

The project repository is organized as follows (paths relevant to the final pipeline; auxiliary IDE/version-control folders omitted):

Path	Purpose
configs/config.yaml	Central YAML configuration (model names, LoRA hyperparameters, embedding source, retrieval/top-K sweep, LLM-baseline backend, generation settings, paths).
data/raw/Spider/	Raw Spider benchmark files (dev.json, tables.json, per-database SQLite files).
data/processed/	Normalized JSONL datasets: spider_eval.jsonl, retrieval_corpus.jsonl, spider_mongo_converted.jsonl, doc/program/commit_generation.jsonl.
data/indices/	Persisted FAISS index (faiss_semantic.index) and the AST/structural store (ast_store.pkl, 1,034 items), both actively used for hybrid RAG retrieval.
src/config.py	Typed dataclass configuration loader (paths, LoRA, embedding, RAG, generation, evaluation).
src/data/data_loader.py	Generic JSON/JSONL dataset loader (DatasetLoader).
src/embeddings/	CodeEmbedder (external BGE embeddings), CodeLMEmbedder (small-code-LM embeddings), SemanticIndexManager (FAISS), ASTIndexer (structural parsing for Python and SQL), and HybridRetriever (blended semantic + structural search).

Path	Purpose
src/models/load_model.py	ModelLoader: base model/tokenizer loading, LoRA setup and training loop, base-vs-LoRA generation.
src/generators/	Task-specific generation wrappers: program_generator.py, doc_gen.py, commit_gen.py, text_to_sql.py, sql_generator.py, and llm_baseline.py (LargeLLMGenerator, the LLM-comparison generator).
src/rag/rag_pipeline.py	RAGPipeline: retrieve_context(), build_prompt(), generate_with_rag() — backed by HybridRetriever.
src/evaluation/	metrics.py, evaluation.py, comparator.py, sql_eval.py, visualization.py, and rag_evaluation.py (RAGEvaluator — RAG-gain measurement and top-K sweep).
src/scripts/	convert_spider_to_jsonl.py, prepare_processed_data.py — one-off data preparation utilities.
output/plots/, src/plots/	Generated evaluation charts (per-model metrics, baseline-vs-fine-tuned comparisons, top-K gain sweep, full model comparison including the LLM baseline).
outputs/generated/	Saved generation samples and metric summaries (baseline_model_results.json, finetuned_model_results.json, per-task generated_samples.json, timestamped text-to-SQL runs).
models/checkpoints/lora_finetuned/	Saved LoRA adapter weights and tokenizer files.
main.ipynb	End-to-end Collab-oriented pipeline notebook (setup, fine-tuning, hybrid indexing, generation, evaluation, RAG-gain measurement, top-K sweep, and LLM-baseline comparison in one run).
baseline_model.ipynb	Loads only the pre-trained base model and evaluates it on text-to-SQL and documentation generation.
finetuned_model.ipynb	Loads/trains the LoRA adapter and evaluates the fine-tuned model on the same two tasks, then compares it to the baseline.
eval.ipynb	Full-scale (1,034-example) Spider evaluation of the example rule-based prediction file: exact-match and SQL execution accuracy.

12. Future Work

- Re-run the RAG-gain measurement and top-K sweep on the full 1,034-example Spider development set rather than the current five-query slice, to obtain a statistically reliable verdict on hybrid RAG's effect for this task.
- Complete the LLM-baseline credential configuration so that the small-code-LM-vs-LLM comparison exercises a genuinely external, larger LLM (for example GPT-5.4-mini or Gemini-3.7-flash) rather than the local fallback observed in this run.
- Adopt a secrets-manager or .env-based credential pattern, with secrets excluded via .gitignore, for all provider API keys before any further sharing of this repository.

- Make the RAG prompt and retrieval corpus task-aware — for example, only retrieving SQL examples for SQL queries, and a separate Python/documentation corpus for those tasks — to address the retrieval-induced domain drift observed in the qualitative samples.
- Scale the text-to-SQL evaluation from the current 100-sample subset to the full 1,034-example Spider development set for both the baseline and fine-tuned models, and add the harder Spider test split.
- Assemble a larger, dedicated instruction-tuning corpus for documentation generation and text-to-SQL — for example, sampled from Spider's 7,000-example training split and a public code/docstring corpus — rather than the current nine-example fine-tuning set.
- Reconcile the project configuration with the checkpoint's actual LoRA target modules, and re-run fine-tuning with the full attention-plus-MLP projection set for a fair comparison against the attention-only run reported here; also correct the configuration file's generation/model nesting.
- Harden the text-to-SQL prompt and output format (for example, function-calling or constrained decoding) to eliminate the residual malformed-output tails and column-mismatch errors observed in some generations.
- Add CodeBLEU and sandboxed Pass@k execution accuracy as first-class reported metrics for the program-synthesis task, using a larger benchmark such as HumanEval or MBPP.

13. Citations and References

- Yu, T., Zhang, R., Yang, K., Yasunaga, M., Wang, D., Li, Z., Ma, J., Li, I., Yao, Q., Roman, S., Zhang, Z., & Radev, D. (2018). Spider: A Large-Scale Human-Labeled Dataset for Complex and Cross-Domain Semantic Parsing and Text-to-SQL Task. EMNLP 2018. arXiv:1809.08887. <https://yale-lily.github.io/spider>
- Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., & Chen, W. (2021). LoRA: Low-Rank Adaptation of Large Language Models. arXiv:2106.09685.
- Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W., Rocktäschel, T., Riedel, S., & Kiela, D. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. NeurIPS 2020.
- Feng, Z., Guo, D., Tang, D., Duan, N., Feng, X., Gong, M., Shou, L., Qin, B., Liu, T., Jiang, D., & Zhou, M. (2020). CodeBERT: A Pre-Trained Model for Programming and Natural Languages. Findings of EMNLP 2020. arXiv:2002.08155.
- Papineni, K., Roukos, S., Ward, T., & Zhu, W. J. (2002). BLEU: a Method for Automatic Evaluation of Machine Translation. ACL 2002.
- Lin, C. Y. (2004). ROUGE: A Package for Automatic Evaluation of Summaries. Text Summarization Branches Out, ACL Workshop.
- Johnson, J., Douze, M., & Jégou, H. (2019). Billion-scale similarity search with GPUs. IEEE Transactions on Big Data. DOI: 10.1109/TBDATA.2019.2921572.
- Hui, B., et al. (2024). Qwen2.5-Coder Technical Report. arXiv:2409.12186. <https://huggingface.co/Qwen/Qwen2.5-Coder-0.5B-Instruct>

- Xiao, S., Liu, Z., Zhang, P., & Muennighoff, N. (2023). C-Pack: Packaged Resources to Advance General Chinese Embedding (BGE embedding family). arXiv:2309.07597. <https://huggingface.co/BAAI/bge-small-en-v1.5>
- Mangrulkar, S., Gugger, S., Debut, L., Belkada, Y., Paul, S., & Bossan, B. (2022). PEFT: State-of-the-art Parameter-Efficient Fine-Tuning methods. Hugging Face. <https://github.com/huggingface/peft>